



Note: As per CakePHP's naming conventions, if the component name contains multiple words, like `RememberMe`, the class name will be `RememberMeComponent` and the file name will be `RememberMeComponent.php`.

Let's continue with our password component example. As the name suggests, our component will be used for generating random passwords in our application.

Now, let's create a file in the `app/Controller/Component` directory and name it `PasswordComponent.php`. The basic structure of our component looks like what's shown below:

```
App::uses('Component', 'Controller');
class PasswordComponent extends Component {
    // some code goes here
}
```



Note: Every component class must extend the main `Component` class. And we need to load the main component class by using the `App::uses('Component', 'Controller')` method.

Now, our basic component is ready. Let's add a function to it that generates a random password string and returns it.

```
App::uses('Component', 'Controller');
class PasswordComponent extends Component {
    public function generate() {
        $length = 10;

        $chars = 'ABCDEFGHJKLNPQRSTU
        VWXYZabcdefghijklmnopqrstuvwxyz' . '0123456789' .
        '-!@#%&*()_+./<?;:[]{}\\|';

        $passwordStr = '';
        $max = strlen($chars) - 1;

        for ($i = 0; $i < $length; $i++) {
            $passwordStr .= $chars[mt_rand(0, $max)];
        }

        return $passwordStr;
    }
}
```

So we have now added our password generator function to the component. When we call this function, it will return a randomly generated password string.

If you see the `generate()` function, it generates a fixed length string of 10 characters. As with any normal PHP function, we can make it more dynamic by adding 'length' as a function parameter, so that we can pass the desired length for our password string.

Let's modify our function to use dynamic length:

```
function generate($length = 8) {
    $chars = 'ABCDEFGHJKLNPQRSTU
    VWXYZabcdefghijklmnopqrstuvwxyz' .
    '0123456789' . '-!@#%&*()_+./<?;:[]{}\\|';

    $passwordStr = '';
    $max = strlen($chars) - 1;

    for ($i = 0; $i < $length; $i++) {
        $passwordStr .= $chars[mt_rand(0, $max)];
    }

    return $passwordStr;
}
```

We have set the default value of `$length` to 8. So, if we don't pass the length, it will generate an eight character long string.

Our final component code should look like what follows:

```
App::uses('Component', 'Controller');
class PasswordComponent extends Component {
    public function generate($length = 8) {
        $chars = 'ABCDEFGHJKLNPQRSTU
        VWXYZabcdefghijklmnopqrstuvwxyz' . '0123456789' .
        '-!@#%&*()_+./<?;:[]{}\\|';

        $passwordStr = '';
        $max = strlen($chars) - 1;

        for ($i = 0; $i < $length; $i++) {
            $passwordStr .= $chars[mt_rand(0, $max)];
        }

        return $passwordStr;
    }
}
```

Now our component is ready to be used in our application's controllers.

How to include the component in the controller

We can include our component in the controller by simply adding its name to the `$components` array. Here, `$components` is a public property of the `Controller` class. Please note that we only need to add the class name of the component, without the 'Component' suffix. For example, to use our `PasswordComponent` in any controller, add the following command:

```
public $components = array('Session', 'Password');
```

After adding this, we can access this component by using `$this->Password`, similar to other components.